

Formulario CMake y CMakeLists

¿Qué es CMake?

CMake es un **Build System Generator**.

NO compila directamente el código.

CMake:

- Lee archivos `CMakeLists.txt`
- Genera archivos de build:
 - Makefiles
 - Ninja
 - Visual Studio Projects
- Configura compiladores, flags y librerías

Flujo General

CMake -> Build System -> Compiler -> Executable

Ejemplo:

- CMake
- Ninja
- g++
- programa final

Build Stages

Stage	Descripción
Configure	CMake analiza CMakeLists.txt
Generate	Genera Makefiles/Ninja/etc
Build	Compilación del código
Link	Une objetos y librerías
Run	Ejecuta el programa

Estructura básica

```
cmake_minimum_required(VERSION 3.20)

project(MyProject)

add_executable(app main.cpp)
```

Comandos básicos

Comando	Descripción
<code>project()</code>	Nombre del proyecto
<code>add_executable()</code>	Crear ejecutable
<code>add_library()</code>	Crear librería
<code>target_link_libraries()</code>	Linkear librerías
<code>target_include_directories()</code>	Agregar includes
<code>find_package()</code>	Buscar librerías instaladas
<code>set()</code>	Crear variables
<code>message()</code>	Imprimir mensajes
<code>option()</code>	Variables booleanas
<code>install()</code>	Instalación del proyecto

Compilación básica

```
cmake -B build
cmake --build build
```

Ejecutar:

```
./build/app
```

Out-of-source builds

Separar código fuente y archivos compilados.

```
project/
|
|-- src/
|-- include/
|-- CMakeLists.txt
|-- build/
```

Ventajas:

- Proyecto limpio
- Fácil borrar builds
- Soporte múltiples configuraciones

Versiones de C++

```
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
```

Modo Debug y Release

```
set(CMAKE_BUILD_TYPE Debug)
```

Tipos:

- Debug
- Release
- RelWithDebInfo
- MinSizeRel

Flags del compilador

```
target_compile_options(app PRIVATE -Wall -Wextra)
```

Ejemplos:

- -Wall
- -Wextra
- -O2
- -g

Agregar múltiples archivos

```
add_executable(app
    src/main.cpp
    src/math.cpp
    src/utils.cpp
)
```

GLOB automático

```
file(GLOB_RECURSE SRC_FILES
    src/*.cpp
)

add_executable(app ${SRC_FILES})
```

Include directories

```
target_include_directories(app PRIVATE include/)
```

Tipos:

- PRIVATE
- PUBLIC
- INTERFACE

Librerías

Static Library

```
add_library(math STATIC math.cpp)
```

Shared Library

```
add_library(math SHARED math.cpp)
```

Linkar librerías

```
target_link_libraries(app PRIVATE math)
```

find_package

Buscar librerías instaladas:

```
find_package(OpenCV REQUIRED)

target_link_libraries(app
    PRIVATE
    OpenCV::OpenCV
)
```

Toolchains

Un Toolchain es el conjunto de herramientas usado para compilar:

- Compiler
- Linker
- Assembler
- Sysroot
- Librerías

Ejemplos:

- GCC
- Clang
- MSVC
- ARM Embedded Toolchain

Toolchain File

Archivo usado para Cross Compilation.

```
cmake -B build \
-DCMAKE_TOOLCHAIN_FILE=toolchain.cmake
```

Ejemplo:

```
set(CMAKE_SYSTEM_NAME Linux)

set(CMAKE_C_COMPILER arm-linux-gcc)
set(CMAKE_CXX_COMPILER arm-linux-g++)
```

Cross Compilation

Compilar en una arquitectura para otra.

Ejemplo:

- PC x86
- Raspberry Pi ARM

Generadores

Generator	Plataforma
Unix Makefiles	Linux
Ninja	Multiplataforma
Visual Studio	Windows
Xcode	macOS

Seleccionar generator:

```
cmake -G Ninja -B build
```

Ninja

Sistema de build rápido y moderno.

```
sudo apt install ninja-build
```

Variables importantes

Variable	Descripción
CMAKE_SOURCE_DIR	Directorio raíz
CMAKE_BINARY_DIR	Directorio build
CMAKE_CXX_COMPILER	Compilador C++
CMAKE_BUILD_TYPE	Debug/Release

Mensajes

```
message("Hola")
message(STATUS "Configurando")
message(WARNING "Advertencia")
message(FATAL_ERROR "Error")
```

Testing

```
enable_testing()

add_test(NAME test1 COMMAND app)
```

Ejecutar tests:

```
ctest
```

Instalación

```
install(TARGETS app
        DESTINATION bin
)
```

CMake Presets

Archivo:

CMakePresets.json

Ejemplo:

```
{
  "version": 3,
  "configurePresets": [
    {
      "name": "debug",
      "binaryDir": "build",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Debug"
      }
    }
  ]
}
```

Usar preset:

```
cmake --preset debug
```

Debug de CMake

```
cmake --trace
cmake --debug-output
```

Estructura moderna recomendada

```
project/
|
|-- src/
|-- include/
|-- tests/
|-- external/
|-- CMakeLists.txt
|-- CMakePresets.json
```

Flujo moderno recomendado

```
cmake -G Ninja -B build

cmake --build build

./build/app
```

Integración con IDEs

- VSCode
- CLion
- Visual Studio
- Qt Creator

Los IDEs normalmente:

- Ejecutan CMake automáticamente
- Configuran Debuggers
- Detectan cambios

Ejemplo de CMakeLists Complejo

Estructura del proyecto:

```
project/
|
|-- src/
|   |-- main.cpp
|   |-- math/
|       |-- math.cpp
|
|-- include/
|   |-- math/
|       |-- math.hpp
|
|-- tests/
|
|-- external/
|
|-- CMakeLists.txt

#####
# Version minima requerida de CMake
#
# IMPORTANTE:
# Algunas features modernas solo existen
# en versiones recientes.
#####

cmake_minimum_required(VERSION 3.20)

#####
# Nombre del proyecto
#
# Tambien define variables internas:
#
# PROJECT_NAME
# PROJECT_SOURCE_DIR
#
# Puede incluir:
# VERSION
# DESCRIPTION
# LANGUAGES
#####

project(
    ComplexProject
    VERSION 1.0
    DESCRIPTION "Ejemplo avanzado de CMake"
    LANGUAGES CXX
)

#####
# Definir el standard de C++
#
```

```

# REQUIRED ON:
# Obliga a usar exactamente ese standard.
#
# Si OFF:
# Puede degradarse a versiones anteriores.
#####

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

#####
# Export compile commands
#
# Genera compile_commands.json
#
# Util para:
# - clangd
# - LSPs
# - herramientas de analisis
#####

set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

#####
# Tipo de build
#
# Debug:
# - simbolos de debug
# - sin optimizacion
#
# Release:
# - optimizaciones
#
# RelWithDebInfo:
# - optimizado + debug
#####

if(NOT CMAKE_BUILD_TYPE)
    set(CMAKE_BUILD_TYPE Debug)
endif()

#####
# Buscar todos los archivos cpp
#
# GLOB_RECURSE:
# busca recursivamente.
#
# Ventaja:
# no agregar archivos manualmente.
#
# Desventaja:
# algunos proyectos prefieren declarar
# explicitamente todos los archivos.
#####

file(
    GLOB_RECURSE
    SRC_FILES
    src/*.cpp
)

#####
# Crear libreria propia

```



```

#
# STATIC:
# libreria estatica.
#
# Otras opciones:
#
# SHARED:
# libreria dinamica (.so/.dll)
#
# INTERFACE:
# libreria solo headers.
#####

add_library(
    mathlib
    STATIC
    src/math/math.cpp
)

#####
# Includes de la libreria
#
# PUBLIC:
# targets que usen mathlib
# heredaran este include.
#
# PRIVATE:
# solo mathlib lo usa.
#
# INTERFACE:
# solo consumidores lo usan.
#####

target_include_directories(
    mathlib
    PUBLIC
    include/
)

#####
# Crear ejecutable principal
#####

add_executable(
    app
    src/main.cpp
)

#####
# Linkear libreria propia
#
# app depende de mathlib.
#
# CMake automaticamente:
# - ordena compilacion
# - agrega linker flags
# - propaga includes PUBLIC
#####

target_link_libraries(
    app
    PRIVATE
    mathlib

```

```

)

#####
# Warnings del compilador
#
# GNU/Clang:
# -Wall
# -Wextra
#
# MSVC usa otros flags.
#####

target_compile_options(
    app
    PRIVATE
    -Wall
    -Wextra
)

#####
# Buscar libreria externa
#
# REQUIRED:
# si no existe -> error.
#
# Otras librerias:
# - OpenCV
# - Boost
# - Eigen3
# - fmt
#####

find_package(fmt REQUIRED)

#####
# Linkear libreria externa
#
# fmt::fmt es un target exportado
# por la libreria.
#
# IMPORTANTE:
# modern CMake prefiere targets
# en lugar de paths manuales.
#####

target_link_libraries(
    app
    PRIVATE
    fmt::fmt
)

#####
# Definir macros de preprocesador
#
# DEBUG_MODE quedara disponible:
#
# #ifdef DEBUG_MODE
#####

target_compile_definitions(
    app
    PRIVATE
    DEBUG_MODE

```

```

)

#####
# Testing
#
# Habilita soporte para ctest.
#####

enable_testing()

#####
# Crear ejecutable de tests
#####

add_executable(
    tests
    tests/test_main.cpp
)

#####
# Linkar tests con libreria
#####

target_link_libraries(
    tests
    PRIVATE
    mathlib
)

#####
# Registrar test
#
# ctest ejecutara este binario.
#####

add_test(
    NAME math_test
    COMMAND tests
)

#####
# Instalacion
#
# DESTINATION:
# donde se instalara.
#
# Ejemplo:
# /usr/local/bin
#####

install(
    TARGETS app
    DESTINATION bin
)

#####
# Mensajes informativos
#####

message(STATUS "Build type: ${CMAKE_BUILD_TYPE}")
message(STATUS "Compiler: ${CMAKE_CXX_COMPILER}")

#####

```

```

# Diferentes compiladores
#
# GNU:
# gcc/g++
#
# Clang:
# clang/clang++
#
# MSVC:
# Visual Studio
#####

if(CMAKE_CXX_COMPILER_ID STREQUAL "GNU")

    message(STATUS "Usando GCC")

elseif(CMAKE_CXX_COMPILER_ID STREQUAL "Clang")

    message(STATUS "Usando Clang")

elseif(CMAKE_CXX_COMPILER_ID STREQUAL "MSVC")

    message(STATUS "Usando MSVC")

endif()

#####
# Cross compilation
#
# Si se usa:
#
# cmake -DCMAKE_TOOLCHAIN_FILE=...
#
# entonces CMake puede compilar
# para otra arquitectura.
#
# Ejemplo:
# x86 -> ARM
#####

message(
    STATUS
    "System processor: ${CMAKE_SYSTEM_PROCESSOR}"
)

#####
# Custom commands
#
# Ejecutar comandos personalizados.
#
# Ejemplo:
# copiar assets
# generar codigo
# ejecutar scripts
#####

add_custom_command(
    TARGET app
    POST_BUILD

    COMMAND ${CMAKE_COMMAND} -E echo
    "Build finalizado"
)

```

```
#####
# Custom targets
#
# Targets virtuales.
#
# Ejemplo:
# make clean_extra
#####

add_custom_target(

    print_info ALL

    COMMAND ${CMAKE_COMMAND} -E echo
    "Proyecto compilado correctamente"

)

#####
# Subdirectorios
#
# Util para proyectos grandes.
#
# Cada carpeta puede tener
# su propio CMakeLists.txt
#####

# add_subdirectory(external)
# add_subdirectory(src/math)

#####
# Presets modernos
#
# Se configuran en:
#
# CMakePresets.json
#
# Facilitan:
# - debug
# - release
# - cross compile
# - CI/CD
#####
```